P.Vishwa nandan(192473015)

PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION

TASK B1 – IMPLEMENT AND INSTRUMENT THE FILE STORE (7 MARKS)

1. Aim

To implement a bus-pass file storage system using Java BufferedReader and BufferedWriter and a buffered attendance-log line counter.

The implementation must support adding, finding, and listing bus passes while safely handling damaged records. It must also measure the performance of buffered and unbuffered reading and analyse search performance as the number of records increases.

2. Problem Description

The university transport office stores bus-pass information in a text file.

Each record contains:

1. Pass ID
2. Student Register Number
3. Student Name
4. Route Number
5. Amount Paid

The application must perform the following operations:

- Add a new bus-pass record.
- Find a pass using its Pass ID.
- List all passes belonging to a particular route.
- Skip damaged records without stopping the search.
- Measure the effect of buffering on file-reading performance.
- Measure search time as the number of records increases.

Note: The supplied Part B starter code contains five fields and does not include the validity date from the full A1 design. The implementation below follows the supplied five-field template so that its required expected output remains unchanged. The full A1 format can be extended by adding the validity date as a sixth field.

3. Task B1.1 – Validate the Implementation (2 Marks)

3.1 Java Program – PassStore.java

3.2 Explanation of the Program

Step 1 – Add a Pass

The add() method accepts the five pass details and writes them to the file using BufferedWriter.

The file is opened in append mode, so previously stored records are not deleted.

Step 2 – Find a Pass

The find() method reads the file line by line using BufferedReader.

It parses each line and checks whether it contains exactly five valid fields.

If the Pass ID matches, the method returns the record.

Otherwise, it continues searching.

Step 3 – List Passes by Route

The listByRoute() method reads the complete file and compares the route number of each valid record with the requested route.

Matching records are added to an ArrayList and returned.

Step 4 – Handle Damaged Records

The isValidRecord() method checks:

- Whether exactly five fields exist.
- Whether the Pass ID is present.
- Whether the Register Number is present.
- Whether the Name is present.
- Whether the Route Number is numeric.
- Whether the Amount Paid is a valid non-negative number.

Invalid records are skipped.

Step 5 – Handle Names Containing Commas

The csvField() method encloses fields containing commas or double quotes in quotation marks.

The parseCsvLine() method understands those quoted fields and prevents the name from being incorrectly divided into multiple fields.

3.3 Expected Output

[P1003, 192311003, Divya, 7, 3800.0]

null

Passes on Route 12:

[P1001, 192311001, Arun, 12, 4500.0]

3.4 Why Does the Damaged P1002 Line Not Stop the Search?

The damaged record contains only two fields:

P1002,DAMAGED LINE

However, a valid record must contain exactly five fields.

The isValidRecord() method detects that the record is incomplete and returns false.

The continue statement skips the damaged record and moves to the next line.

Therefore, the damaged P1002 line does not stop the search for P1003.

4. Test Case Validation

The following cases verify the implementation.

Test Case	**Input / Condition**	**Expected Result**
TC1	Add a valid pass P1001	Pass is added
TC2	Find P1001	Matching record is returned
TC3	Find P9999	Returns null
TC4	Find P1003 after a damaged line	P1003 is found successfully
TC5	Search for damaged P1002	Returns null
TC6	List route 12	Only route 12 records are returned
TC7	Name contains a comma	Name is stored and read correctly
TC8	Amount contains letters	Damaged record is skipped
TC9	File does not exist	File-not-found IOException is reported
TC10	Empty line exists	Empty line is skipped

5. Task B1.2 – Compare with Your Design (1 Mark)

5.1 Comparison With Part A Pseudocode

The implementation follows the main operations described in Part A.

Part A Design	Part B Implementation
Add a new pass	Implemented using BufferedWriter
Find pass by ID	Implemented using BufferedReader
List passes by route	Implemented using listByRoute()
Check field count	Implemented using isValidRecord()
Skip damaged records	Implemented using continue
Handle missing files	IOException is reported
Handle names containing commas	CSV quoting and parsing implemented

5.2 Changes Made During Implementation

Some improvements were made during implementation.

1. Try-with-resources was used to close BufferedReader and BufferedWriter automatically.
2. A CSV parser was added to correctly handle names containing commas.
3. The program checks numeric route and amount fields before accepting a record.
4. The supplied TODO code supports five fields, so the implementation follows the supplied format rather than adding the validity-date field.
5. A separate listByRoute() method was added to complete all three required operations.

Conclusion: The implementation follows the original design while improving record validation and CSV handling.

6. Task B1.3 – Measure the Effect of Buffering (2 Marks)

6.1 Objective

To compare the time taken by unbuffered and buffered file-reading techniques for attendance logs of different sizes.

The following file sizes are tested:

- 1 MB
- 10 MB
- 50 MB

The program counts the number of newline characters and measures the execution time using System.nanoTime().

6.2 Unbuffered Reading

The unbuffered program uses:

```
FileInputStream input =
    new FileInputStream(file);


int data;


while ((data = input.read()) != -1) {


    // Process one byte
}
```

It reads one byte per read call.

Therefore, a 50 MB file requires approximately 50 million data-byte read calls.

6.3 Buffered Reading

The buffered program reads 64 KB blocks.

```
BufferedInputStream input =
    new BufferedInputStream(
        new FileInputStream(file),
        64 * 1024
    );
```

Multiple bytes are read in one operation, reducing repeated read overhead.

6.4 Java Program – LineCounterBenchmark.java

6.5 Program Explanation

Step 1 – Generate Test Files

The program generates three attendance logs of different sizes.

Step 2 – Count Lines Without Buffering

The unbuffered method reads one byte at a time using FileInputStream.

Step 3 – Count Lines With Buffering

The buffered method reads 64 KB blocks using BufferedInputStream.

Step 4 – Check Correctness

Both methods must return the same line count.

Step 5 – Measure Execution Time

System.nanoTime() records the time before and after each operation.

Step 6 – Calculate Median

Three measurements are taken for each method, and the median is displayed to reduce the effect of unusually slow or fast runs.

6.6 Experimental Results Table

Run LineCounterBenchmark.java on your laptop and record the actual results.

File Size	Unbuffered Median	Buffered Median
1 MB	Record actual time	Record actual time
10 MB	Record actual time	Record actual time
50 MB	Record actual time	Record actual time

The timings are system-dependent and should not be replaced with invented values.

6.7 Interpretation

The unbuffered version performs one-byte read operations and creates more repeated read overhead.

The buffered version reads multiple bytes together, significantly reducing low-level read operations.

As file size increases, the performance difference is generally more noticeable.

However, actual execution times depend on:

- CPU speed.
- Storage device.
- Operating-system caching.
- Java version.
- Available memory.
- Background processes.

The measured values should be compared with the prediction in A2.1.

7. Task B1.4 – Measure Growth (2 Marks)

7.1 Objective

To measure the time required to find a bus pass as the number of records increases.

The following record counts are tested:

- 1,000 records.
- 10,000 records.
- 50,000 records.
- 100,000 records.

The last record is searched each time to demonstrate the worst-case sequential-search behaviour.

7.2 Java Program – PassFindBenchmark.java

7.3 Program Explanation

Step 1 – Generate Pass Records

The program creates files containing different numbers of bus-pass records.

Step 2 – Select the Last Record

The search target is always the last record.

This ensures that the program must read almost the entire file before finding the requested pass.

Step 3 – Warm-Up

Two searches are performed before timing to reduce the influence of initial JVM execution overhead.

Step 4 – Measure Search Time

Seven searches are performed for each file size.

The median execution time is calculated.

Step 5 – Display Results

The program prints the number of records and corresponding median search time.

7.4 Experimental Results Table

Run PassFindBenchmark.java and enter the measured values.

Number of Records	Search Position	Median Search Time
1,000	Last record	Record actual time

10,000	Last record	Record actual time
50,000	Last record	Record actual time
100,000	Last record	Record actual time

7.5 Interpretation of Search Growth

The find() method performs sequential searching.

It starts from the first record and compares the Pass ID with the requested ID.

When the number of records increases, more lines must be read and examined.

Therefore, the search time generally increases as the file grows.

The time complexity is:

O(n)

where n is the number of bus-pass records.

For a successful search of the last record, almost every record must be examined.

A search for the first record may be faster, while a missing Pass ID requires scanning the complete file.

8. Combined Performance Analysis

8.1 Buffered vs Unbuffered Reading

Factor	**Unbuffered**	**Buffered**
Read size	One byte	Multiple bytes
Number of low-level read operations	High	Much lower
Memory usage	Very low	Fixed buffer memory
Large-file performance	Generally slower	Generally faster
Suitability	Small or special reads	Large sequential files

8.2 Sequential Search Growth

Records	**Expected Search Behaviour**
1,000	Examines up to 1,000 records
10,000	Examines up to 10,000 records
50,000	Examines up to 50,000 records

100,000	Examines up to 100,000 records

The search time grows with the number of records because the application scans the file sequentially.

8.3 Overall Engineering Observation

Buffering improves file-reading efficiency because it reduces repeated read operations.

However, buffering does not change the asymptotic time complexity of sequential search.

Finding a pass by ID still requires O(n) time when no index is maintained.

For applications with frequent searches, a HashMap index or database can provide faster record lookup.

9. Final Result

The bus-pass file store was implemented using BufferedReader and BufferedWriter.

The implementation supports adding records, finding passes by ID, and listing records by route number.

The supplied TODO in PassStore.java was completed, and damaged records were skipped without stopping the search.

A buffered attendance-log line counter was implemented and compared with an unbuffered version using files of 1 MB, 10 MB, and 50 MB.

The time required to find a pass was also measured for files containing 1,000, 10,000, 50,000, and 100,000 records.

The measurements demonstrate the benefits of buffering and the linear growth of sequential searching.

Thus, the implementation validates the file-storage design and provides experimental evidence of its performance.